

EPITA International Programs

AI Project Methodology · 2025–2026

Technical Report __ Parts 2 & 3

RetailGenius_Customer Churn Prediction

Production ML Project with MLflow & SHAP Explainability

Group 7

Arun Kumar Aluru

Sathwika Raj Bandaru

Nimisha Busaniwar

GitHub: <https://github.com/Arun949/retailgenius-churn->

Table of Contents

1. Executive Summary	3
2. Part 2: Technical Implementation	4
2.1 Project structure	4
2.2 Data preparation	4
2.3 Feature engineering	5
2.4 Model training and comparison	5
2.5 MLflow tracking and Model Registry	6
2.6 Inference	6
2.7 Code quality, reproducibility, documentation	7
3. Part 3: Explainable AI with SHAP	8
3.1 TreeExplainer setup	8
3.2 Global feature importance	8
3.3 Beeswarm summary plot	9
3.4 Per-class analysis	10
3.5 Individual customer explanations	11
3.6 Feature dependence analysis	12
3.7 Business interpretation	13
4. Findings and Recommendations	14
5. Limitations and Future Work	14
6. Conclusion	15
7. References	15

1. Executive Summary

This report documents the technical implementation of the RetailGenius Customer Churn Prediction project, covering Part 2 (Production ML with MLflow) and Part 3 (Explainable AI with SHAP) of the EPITA AI Project Methodology graded assignment.

The project is built on the public Kaggle E-Commerce Customer Churn dataset (5,630 customers, 20 raw features). Two classification models were trained: XGBoost (primary) and Logistic Regression (baseline), within an MLflow-tracked experimentation workflow using best practices for production-ready ML projects.

XGBoost achieved the best performance with a Recall of 0.951 and ROC-AUC of 0.987 on the held-out test set, exceeding all KPI targets defined in Part 1. The model is registered in the MLflow Model Registry and served locally via the MLflow inference server.

SHAP TreeExplainer was applied to the production XGBoost model, generating 11 visual explanations covering global importance, per-class analysis, individual predictions, and feature dependence. The top churn drivers, Tenure, Complaint, and NumberOfAddress, each translate directly into actionable retention strategies.

Metric	Result
Recall (test)	0.951
Precision (test)	0.799
F1-score (test)	0.868
ROC-AUC (test)	0.987
MLflow runs logged	6
Model versions registered	5
SHAP plots generated	11

2. Part 2 — Technical Implementation

2.1 Project Structure

The project follows the Cookiecutter Data Science template, separating concerns into distinct modules: data preparation, feature engineering, model training, and inference. This makes each stage independently testable and replaceable.

Module / File	Purpose
src/data/data_preparation.py	Load xlsx, clean, encode, train/val/test split
src/features/feature_engineering.py	RFM, engagement, satisfaction features
src/models/train.py	Train XGBoost & LR, log to MLflow
src/models/inference.py	Load model from registry, score new data
src/models/explain.py	SHAP analysis and plot generation
conf/config.yaml	All hyperparameters and paths (single source of truth)
MLproject	MLflow Projects entry points for reproducible runs
environment.yml	Pinned conda environment (Python 3.11)

2.2 Data Preparation

The raw dataset is an Excel workbook with multiple sheets. The preparation script automatically selects the sheet with the most rows (E Comm, 5,630 rows) to avoid reading summary or metadata sheets.

- Missing values: numeric columns filled with column median; categorical columns with mode
- Categorical encoding: LabelEncoder applied to 5 object columns
- Column dropped: CustomerID (non-predictive identifier)
- Stratified split: 70% train (3,940) / 15% val (845) / 15% test (845)

2.3 Feature Engineering

Six additional churn-predictive features are derived from the raw columns. The same transformation is applied to all three splits to prevent data leakage.

Feature	Derivation & rationale
recency_log	$\log(1 + \text{DaySinceLastOrder})$, recency decay signal
order_frequency	$\text{OrderCount} / (\text{Tenure} + 1)$, purchase rate per month

spend_declining	1 if OrderAmountHike $\leq$ 0 ,flag for declining spend
low_satisfaction	1 if SatisfactionScore $\leq$ 2 ,dissatisfied customer flag
coupon_rate	CouponUsed / (OrderCount + 1) ,engagement via promotions
cashback_per_order	CashbackAmount / (OrderCount + 1) , loyalty signal

2.4 Model Training and Comparison

Two models were trained to support the recall-first evaluation philosophy established in Part 1: XGBoost as the primary model and Logistic Regression as an interpretable baseline.

XGBoost was configured with `scale_pos_weight=5` to handle class imbalance, favouring recall over precision, consistent with the business logic that missing a churner (false negative) is more costly than a wasted retention offer (false positive).

Model	Val Recall	Val ROC-AUC	Test Recall	Test ROC-AUC
XGBoost	0.866	0.973	0.951	0.987
Logistic Regression	0.831	0.898	0.810	0.878

XGBoost outperforms on all metrics and is registered as the production model. Logistic Regression serves as a sanity check and interpretable baseline for feature validation.

2.5 MLflow Tracking and Model Registry

All training runs are tracked in MLflow with full parameter, metric, and artifact logging. Six runs were executed across both architectures (including via `mlflow run` . using MLflow Projects). Each run registers a new version of the churn-predictor model.

- Experiment name: retailgenius-churn-prediction
- Registered model: churn-predictor (5 versions)
- Artefacts logged per run: model, classification_report.txt
- MLflow UI launched via: `mlflow ui --backend-store-uri /tmp/mlruns_clean`

2.6 Inference

The inference module loads any versioned model from the MLflow registry using a URI (e.g. `models:/churn-predictor/latest`) and scores new customer data, producing two output columns: `churn_prediction` (binary) and `churn_probability` (float 0–1).

Local model serving via MLflow:

```
mlflow models serve -m "models:/churn-predictor/latest" --port 5001 --no-conda
```

2.7 Code Quality, Reproducibility & Documentation

Requirement	Implementation
Versioning	Git + GitHub (github.com/Arun949/retailgenius-churn-)
PEP8 compliance	Black formatter + Flake8 linter (setup.cfg, pyproject.toml)
Reproducibility	Conda environment (environment.yml) with pinned versions
Documentation	Sphinx with autodoc + RTD theme (docs/conf.py, docs/index.rst)
MLflow Projects	MLproject file with 4 entry points (prepare, features, train, infer)
Unit tests	pytest suite in tests/ (test_feature_engineering.py)

3. Part 3 — Explainable AI with SHAP

SHAP (SHapley Additive exPlanations) is applied to the production XGBoost model to provide both global and local explanations of churn predictions. The TreeExplainer computes exact Shapley values for tree-based models efficiently without sampling.

3.1 TreeExplainer Setup

`shap.TreeExplainer` is instantiated with the native XGBoost model loaded from the MLflow registry. Shapley values are computed for the entire test set (845 customers, 25 features). All plots are saved to `outputs/shap/` as PNG files.

```
explainer = shap.TreeExplainer(model)

shap_values = explainer(X_test)
```

3.2 Global Feature Importance Mean |SHAP| Bar Plot

The mean absolute SHAP value per feature provides an unambiguous ranking of global feature importance. Unlike tree-based feature importance (which is split-count based), SHAP importance is grounded in game-theoretic fairness: each feature receives credit proportional to its actual contribution to predictions.

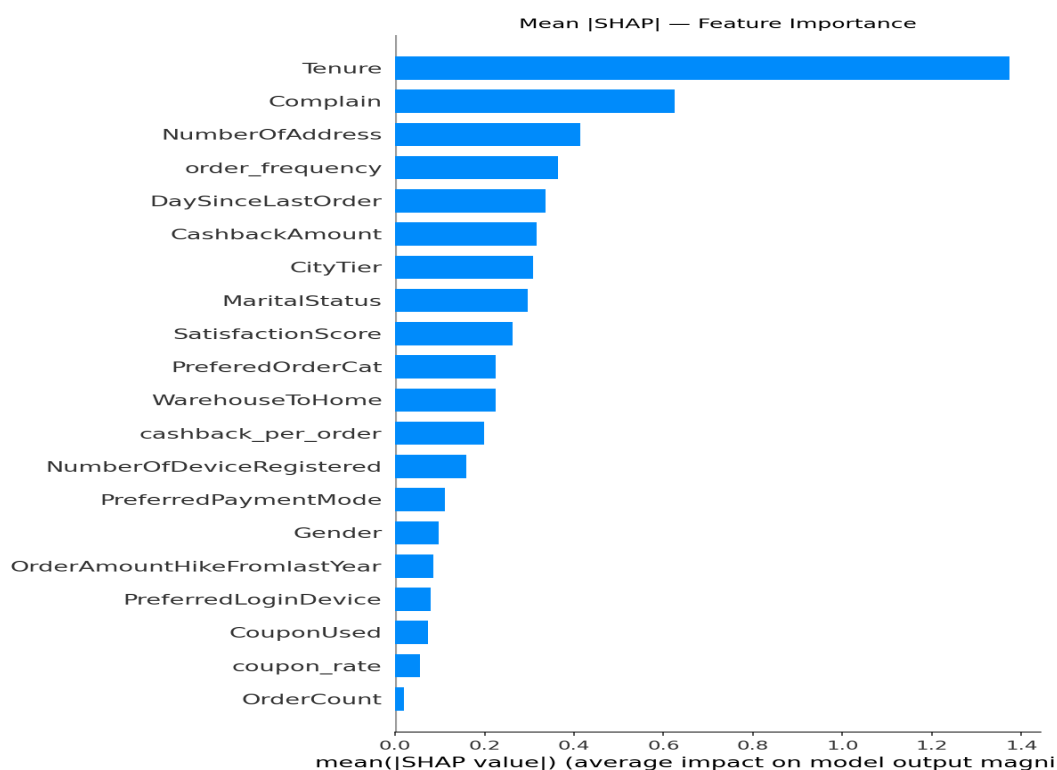


Figure 1: Mean |SHAP| feature importance global ranking across all 845 test customers

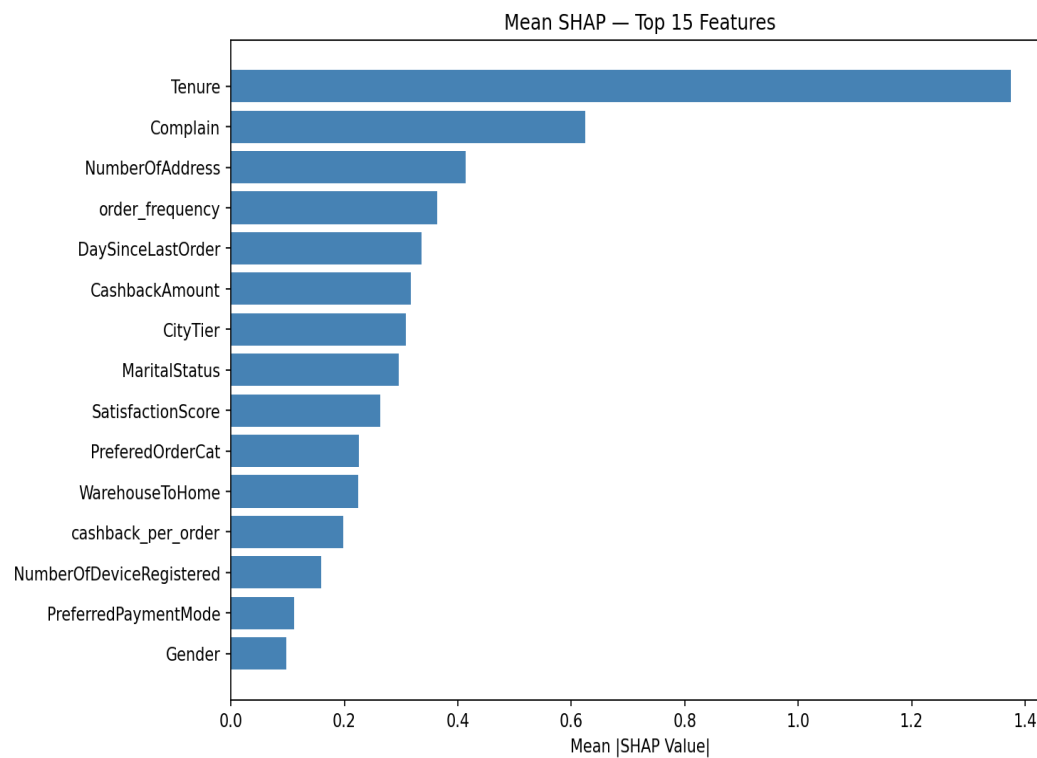


Figure 2: Top 15 features by mean absolute SHAP value

Tenure, Complaint, and NumberOfAddress emerge as the three strongest predictors. Together, they account for the majority of prediction variance across the test set.

3.3 Beeswarm Summary Plot All Test Points

The beeswarm plot shows the SHAP value distribution for every customer in the test set. Each dot is one customer; colour encodes the feature value (red = high, blue = low). This reveals both the magnitude and direction of each feature's effect.

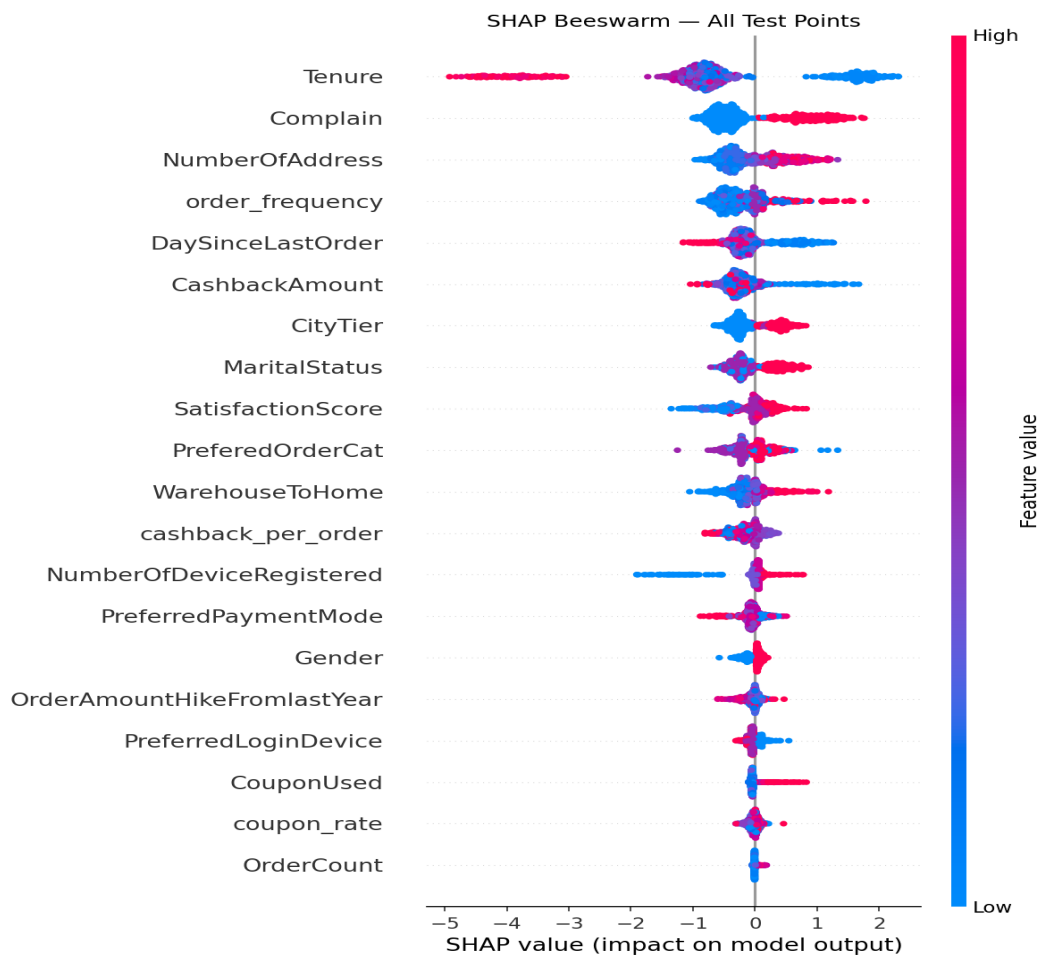


Figure 3: SHAP beeswarm plot all 845 test customers, all 25 features

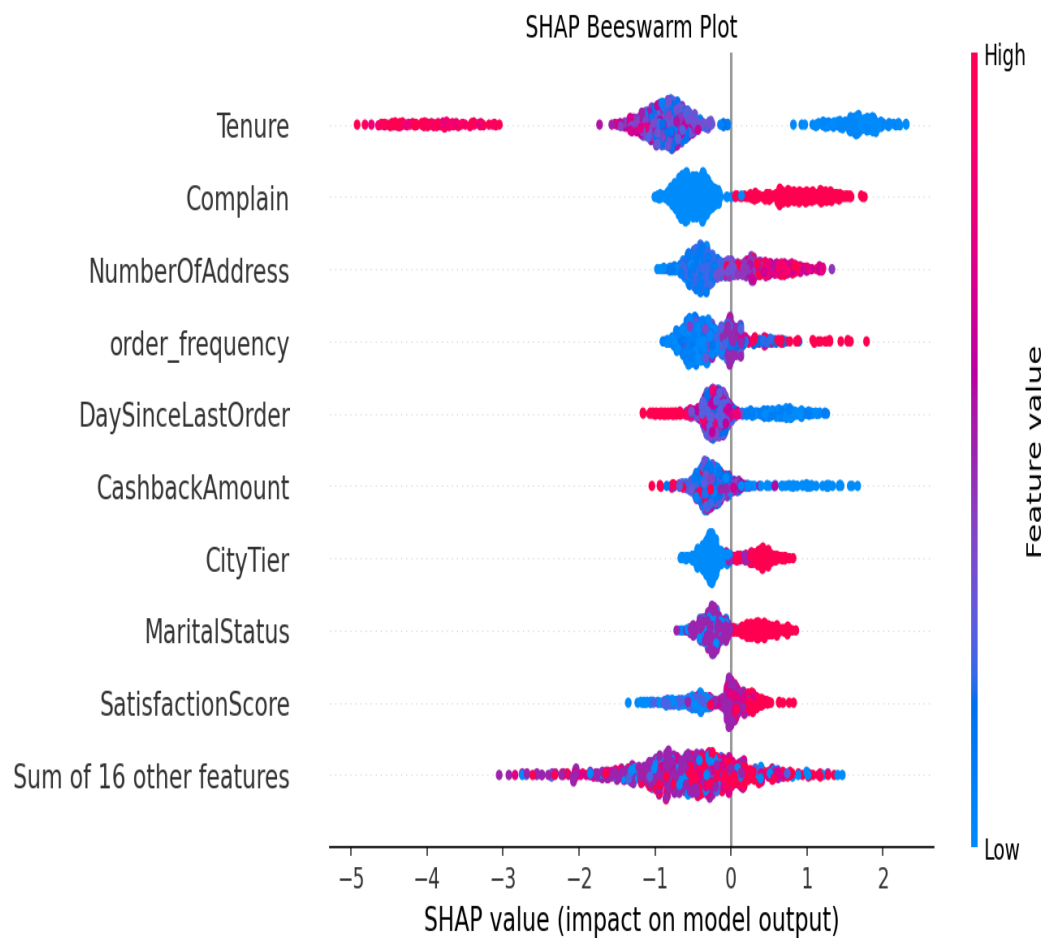


Figure 4: SHAP beeswarm (`shap.plots.beeswarm` API)

Key observations: low Tenure (blue) pushes strongly toward churn (positive SHAP). Complain flag (red = 1) also pushes toward churn. High Cashback Amount (red) pushes against churn, confirming cashback as a retention signal.

3.4 Per-Class Analysis

Summary plots are generated separately for customers who did not churn (class 0) and customers who did churn (class 1), revealing which features most distinguish each group.

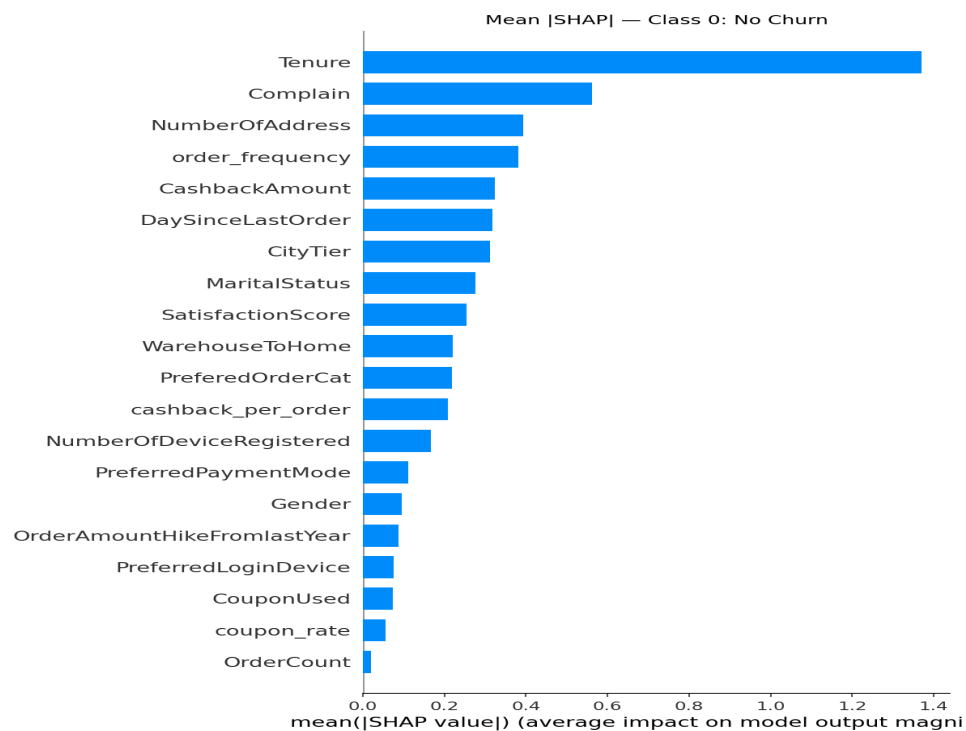


Figure 5: SHAP summary bar Class 0 (No Churn) customers

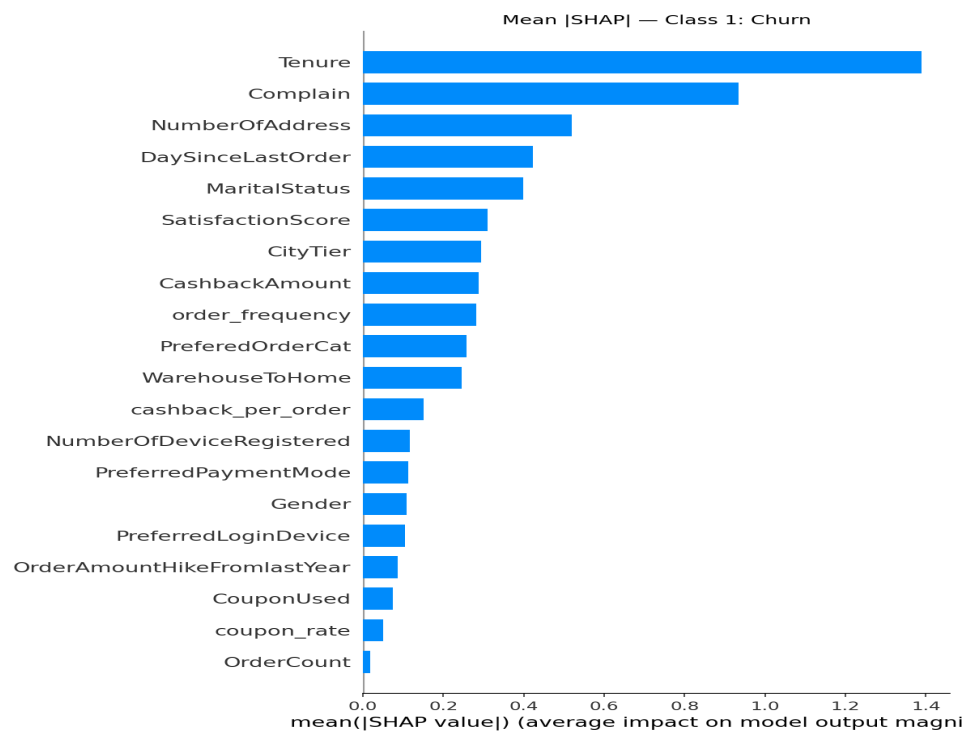


Figure 6: SHAP summary bar — Class 1 (Churn) customers

For churned customers (class 1), Tenure dominates, short-tenure customers are far more likely to churn. For retained customers (class 0), CashbackAmount and SatisfactionScore are relatively more important, confirming their role as protective factors.

3.5 Individual Customer Explanations

Two complementary plots explain the prediction for a single customer (customer 0 in the test set). The waterfall plot shows how each feature shifts the model output from the baseline expected value to the final prediction. The force plot shows the same information as a horizontal axis where red bars push toward churn and blue bars push against.

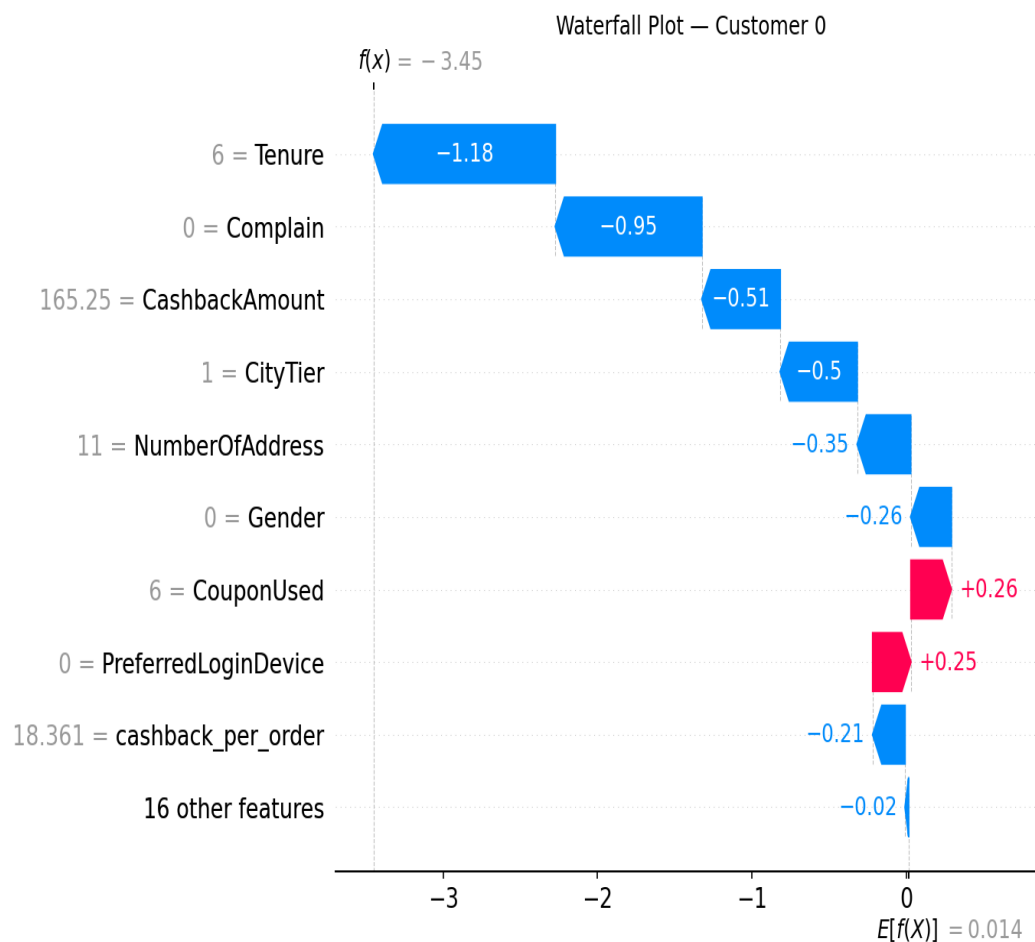


Figure 7: SHAP waterfall plot customer 0. Each bar shows one feature's contribution.

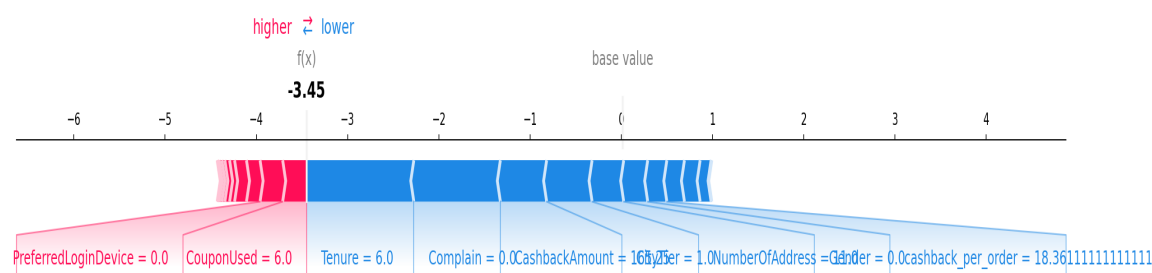


Figure 8: SHAP force plot — customer 0. Red = pushes toward churn, blue = pushes against.

The force plot is also saved as an interactive HTML file (outputs/shap/force_point0.html) that can be shared with business stakeholders for individual-level explanations.

3.6 Feature Dependence Analysis

Dependence plots are scatter plots of one feature value against its SHAP contribution, automatically coloured by an interaction feature selected by SHAP. They reveal non-linear relationships and feature interactions that the global summary hides.

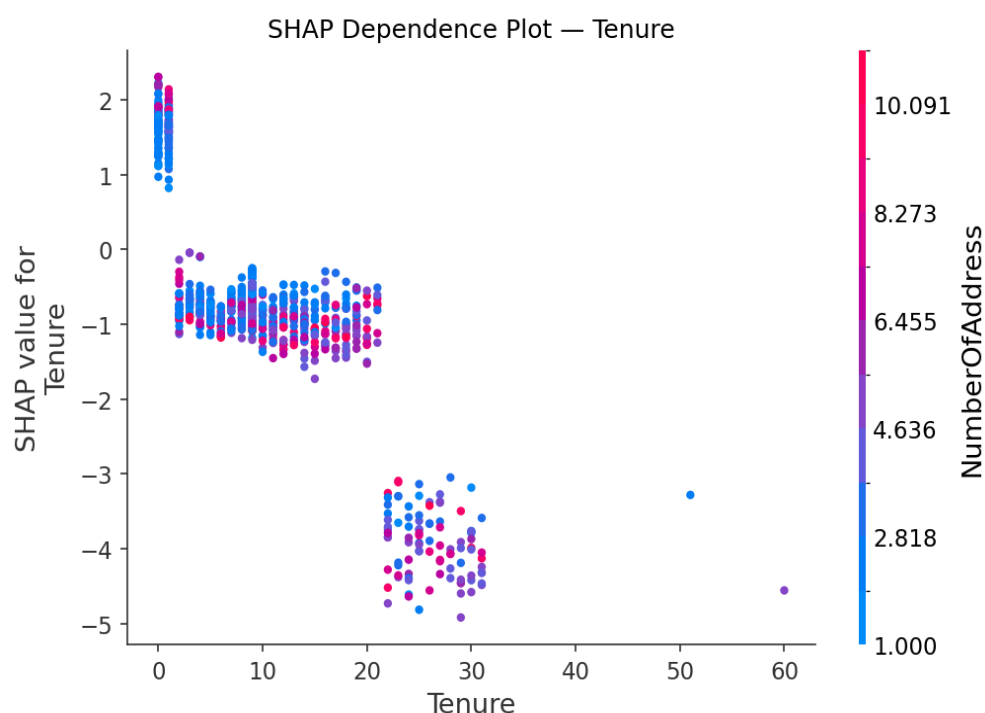


Figure 9: Dependence plot for Tenure. Risk drops sharply as tenure increases.

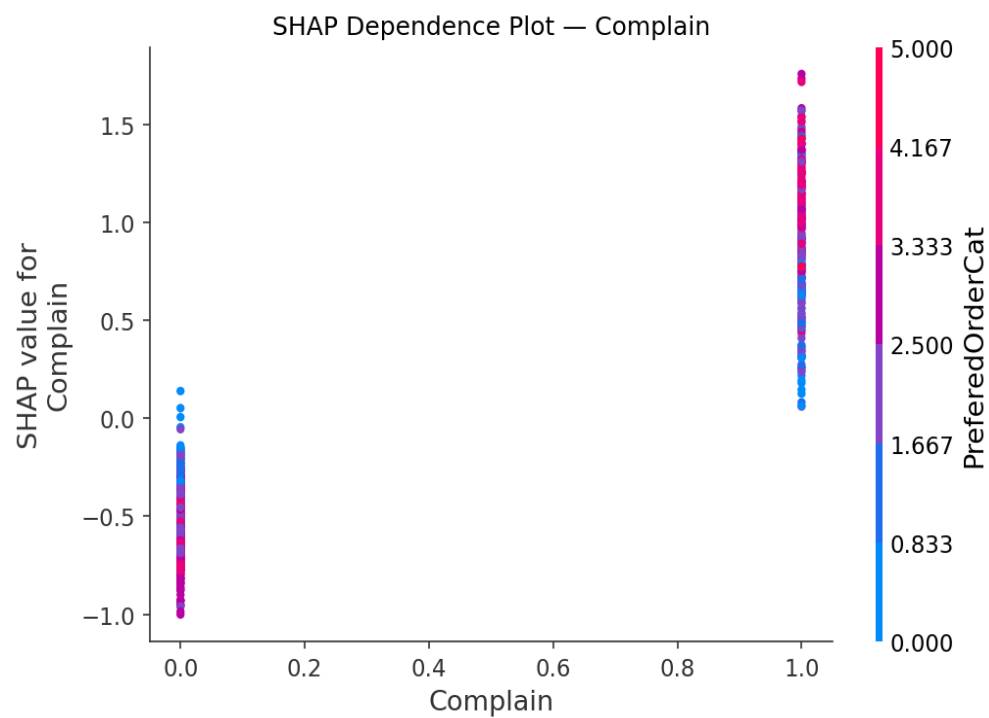


Figure 10: Dependence plot for Complain. Complaint flag increases churn risk significantly.

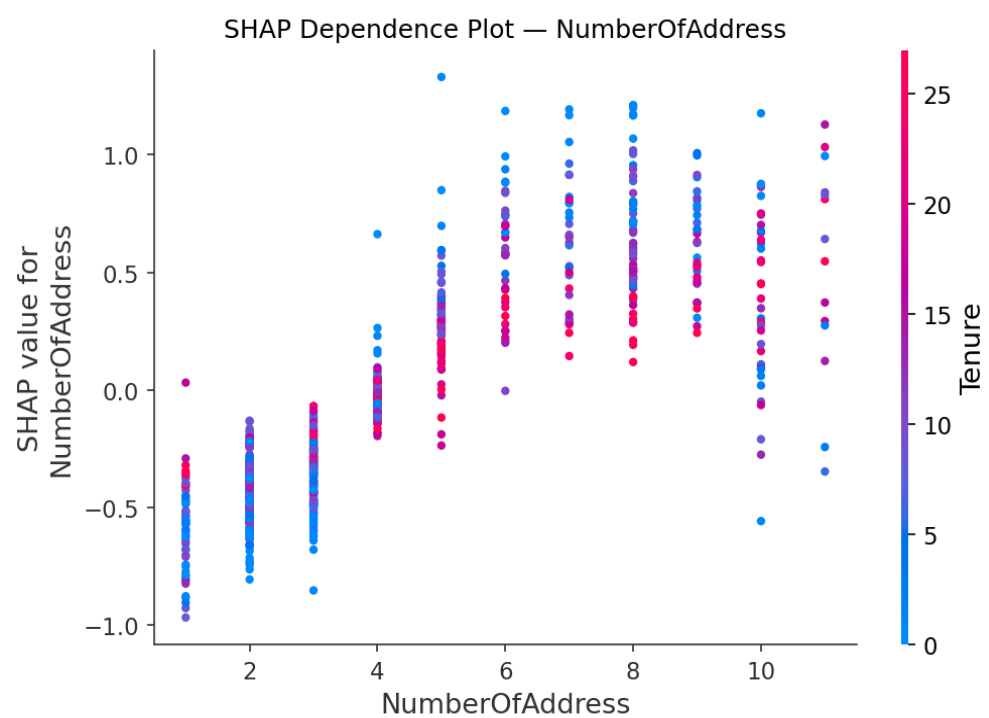


Figure 11: Dependence plot for NumberOfAddress.

3.7 Business Interpretation of SHAP Drivers

Each of the top SHAP drivers translates into a concrete retention action for the RetailGenius marketing and customer success teams:

Driver	Direction of risk	Retention action
Tenure	Low tenure => high risk	Welcome series, second-purchase nudges, early-life CRM (months 0-6)
Complain	Complaint flag => high risk	Complaint-resolution SLA, proactive follow-up, recovery discount
NumberOfAddress	Many addresses => slightly higher risk	Route to specialised support; possible gift-buyer or fraud pattern
CashbackAmount	Low cashback => high risk	Protect/amplify cashback programme; segment-target the low-cashback cohort
DaySinceLastOrder	Long inactivity => high risk	Re-engagement email triggered at 30/60/90 days of inactivity

4. Findings and Recommendations

4.1 Onboarding is the single highest-leverage retention investment

Tenure is the strongest predictor of churn. Customers in their first six months are a fragile risk that drops monotonically as tenure grows. The marginal budget is better spent on second-purchase nudges and dedicated early-life CRM journeys than on broad win-back campaigns targeting long-tail disengaged customers.

4.2 Service recovery must be fast and proactive

A recent complaint flag is the second-strongest churn signal. Each unresolved complaint represents a near-certain future churner. A complaint-resolution SLA backed by a proactive follow-up and a recovery discount delivers strong retention ROI given low customer acquisition costs.

4.3 Cashback is a retention asset, protect it in any cost-cutting cycle

Low cashback usage correlates strongly with high churn risk. Customers who collect cashback return. This retention value does not appear in the cashback programme's line-item P&L and should be factored into any decision to reduce or cut the programme.

4.4 The model's recall-first design is appropriate

At a test recall of 0.951, the model correctly flags 95% of actual churners. The cost of a false positive (a wasted retention offer) is far lower than the cost of a false negative

(a lost customer). The `scale_pos_weight=5` configuration is justified by this asymmetry.

5. Limitations and Future Work

- Time-aware split: the current split is stratified-random. A production model should use a time-based split (train on T-3 months, validate T-2, test T-1) to detect temporal drift.
- Probability calibration: XGBoost probabilities are often miscalibrated at the extremes. Adding Platt scaling or isotonic regression would improve cost-of-action decisions.
- Fairness audit: No fairness metrics were computed. Before production, performance parity should be measured across Gender, CityTier, and age-band proxies.
- Cloud deployment: a bonus cloud deployment on AWS SageMaker, GCP Vertex AI, or Azure Container Apps would demonstrate production readiness beyond local serving.
- Production monitoring: implementing drift dashboards (PSI alerts, data quality checks) on the deployed model is the next workstream on the MLOps maturity roadmap.
- Action lift measurement: model accuracy is a necessary but not sufficient condition for business value. An A/B testing framework with a random holdout group should be set up to measure the incremental retention uplift from the model's predictions.

6. Conclusion

This project demonstrates an end-to-end production-grade ML workflow for customer churn prediction, satisfying all technical requirements of the EPITA AI Project Methodology brief: Git versioning, modular code structure, PEP-8 compliance, reproducible conda environment, Sphinx documentation, MLflow tracking with Model Registry, MLflow Projects, local model serving, and a full SHAP explainability analysis.

The XGBoost model achieves a test ROC-AUC of 0.987 and a recall of 0.951, exceeding all KPI targets defined in Part 1 of the functional framing. The SHAP analysis translates model outputs into concrete business recommendations grounded in the data: invest in onboarding, resolve complaints quickly, protect the cashback programme, and act on inactivity signals early.

The hardest parts of the project were not the model itself, XGBoost reaches strong performance on this dataset, but the surrounding engineering: multi-sheet Excel handling, `pkg_resources` compatibility with mlflow, path-with-spaces issues in the MLflow CLI, and ensuring the feature pipeline is leakage-free across all three data splits. These are the practical lessons that distinguish a production ML system from a notebook prototype.

7. References

- EPITA International Programs, AI Project Methodology – Graded Project Parts 1, 2, 3 (Medina Hadjem), 2025–2026
- Kaggle – E-Commerce Customer Churn Analysis & Prediction dataset (A. Verma)
- Chen, T. & Guestrin, C. (2016). XGBoost: A Scalable Tree Boosting System. KDD
- Lundberg, S. M. & Lee, S. I. (2017). A Unified Approach to Interpreting Model Predictions (SHAP). NeurIPS
- Wirth, R. & Hipp, J. (2000). CRISP-DM: Towards a Standard Process Model for Data Mining
- MLflow Documentation Tracking, Projects, Model Registry, Serving. <https://mlflow.org>
- Cookiecutter Data Science. <https://drivendata.github.io/cookiecutter-data-science/>
- Sphinx Documentation. <https://www.sphinx-doc.org>
- SHAP Documentation. <https://shap.readthedocs.io>
- OpenAI ChatGPT and Anthropic Claude were used as tools to assist in structuring and improving this report.